Reload or Recompute? A Calibrated Rule for KV-Cache Reuse Across Commodity GPUs

M Mohith
Chinmaya Vishwa Vidyapeeth, Kochi, India
achumohith14@gmail.com
ORCID: 0009-0003-2978-3045

Abstract

When a conversation returns, a server must choose whether reloading its key–value (KV) cache or recomputing its prefix gives a faster first token. We evaluate a break-even bandwidth rule for time to first token (TTFT) that includes resident-cache continuation, fixed overhead, and CPU and transfer costs. Building on existing calibration and compute/load analyses, we timestamp and freeze per-cell predictions after short calibration and evaluate separate sweeps on commodity and older GPUs, including short contexts and separate CPU-memory and file paths. Frozen predictions correctly classify 574/636 resolved decisions on L4, 192/206 on A10 x16, and 398/426 on A10G x8. For Qwen2.5-1.5B with a 1,024-token prefix and a 32-token new turn, break-even simulated-bandwidth brackets rise from T4's 100–400 MB/s to 800–1,600 MB/s on L4 and A10 x16. Long-context extrapolation can produce large TTFT errors. We release measurements, frozen predictions, reference code, and a source-linked claim ledger.

Artifacts: github.com/mohi-devhub/reload-or-recompute; frozen release v0.1.0-data.

1 Introduction

In the T4 pilot, Qwen2.5-1.5B returned a first token in a median 73.724 ms after reloading a pinned CPU cache for an 8,192-token prefix and a 32-token new turn; recomputing the same input took 1,737.026 ms. When should a server reload a saved conversation cache, and when should it rebuild the prefix? The answer depends on how much time reuse saves after paying for transfer and continuation.

Returning conversations make the choice between reloading a saved prefix and recomputing it a practical latency question for small-model operators on commodity and older GPUs.

py-kvcache already characterizes the reload-versus-recompute trade-off and its hardware dependence, with offline calibration and bandwidth frontiers [3]. Cake compares GPUs using simulated I/O bandwidth on MHA and GQA models, and overlaps compute and load [2]. We do not claim novelty for KV reuse, offload, break-even thresholds, or choosing compute versus load. Our question is whether a rule fixed during the T4 study can predict later measurements on other GPUs after only short local calibration.

Contributions. The first contribution is timestamped, frozen per-cell predictions evaluated on separate sweeps after short local calibration, including GPUs beyond the T4 used to develop the rule. Main-model calibration takes about a minute. Second, we measure commodity and older GPUs and short contexts where reuse costs can outweigh recomputation. Third, we measure CPU-memory and file paths separately, with physical-read observability. The rule's form is fixed before transfer testing; its costs still require calibration.

The reference path measures one request with a saved cache already available. Choosing whether to persist that cache also requires write cost, capacity, and return probability.

Key takeaways.

- A short local calibration can predict most reload-versus-recompute choices on a new GPU.
- Faster prefill can make recomputation preferable for short prefixes and raise the bandwidth needed for reload.
- Compare the full returning-turn time, and check what the file path actually reads.

2 The TTFT-adjusted decision rule

Let L be the cached prefix length, N the new-turn length, and $S(L)$ the KV payload in bytes. Let $T_{\text{recompute}}(L, N)$ measure fresh prefill of prefix plus new turn through first-token selection. Let $T_{\text{GPU hit}}(L, N)$ measure continuation when the prefix cache is already resident. For a chosen transfer leg with effective bandwidth B, use

$$\widehat{T}_{\text{reload}}(L, N; B) = \widehat{T}_{\text{GPU hit}}(L, N) + \widehat{h} + \frac{S(L)}{B} + \widehat{T}_{\text{other}}(L, N). \tag{1}$$

Here h is fixed overhead, and T_{other} contains measured CPU or transfer legs not represented by S/B. For a file path this may include read and deserialization work, with H2D transfer represented either by S/B or by another measured leg. Costs must be counted once. Direct `torch.load` and staged byte loading have different CPU paths; their cost models must follow those paths.

Writing $D = \widehat{T}_{\text{recompute}} - \widehat{T}_{\text{GPU hit}} - \widehat{h} - \widehat{T}_{\text{other}}$, the break-even bandwidth is

$$B^*(L, N) = \frac{S(L)}{\widehat{T}_{\text{recompute}}(L, N) - \widehat{T}_{\text{GPU hit}}(L, N) - \widehat{h} - \widehat{T}_{\text{other}}(L, N)}, \qquad D > 0. \tag{2}$$

For a positive denominator, reload is predicted faster above B^*. If the denominator is nonpositive, the additive model predicts no finite bandwidth at which that path wins. This is a model decision, not proof that every implementation must lose. Near a zero denominator, small baseline errors cause large threshold errors; we retain uncertainty rather than force a finite crossing.

A prefix-only approximation compares S/B with prefix prefill alone:

$$B^*_{\text{prefix}}(L) = \frac{S(L)}{T_{\text{prefill}}(L)} = k_{\text{KV}}\, r_{\text{prefill}}(L), \tag{3}$$

where k_{KV} is bytes per token and r_{prefill} is measured prefix tokens per second. This approximation helps explain the bandwidth axis. It does not account for the resumed-turn baseline, buffer creation, tensor views, or CPU work. In particular, reusing a short prefix can leave too little saved time to pay those costs.

The rule has two tests. The T4 redesign evaluates descriptive agreement with measured same-session recompute and resident-hit baselines. Cross-GPU transfer instead predicts those baselines and transfer legs from calibration, freezes the predictions, and scores later sweeps. The latter additionally tests baseline prediction and variation between GPU instances.

3 Setup and measurement protocol

3.1 Three phases and observed hardware

The pilot tests the main model on T4 at prefixes 512, 2,048, and 8,192, with 32- and 256-token turns. The pilot has 12 measured repeats per cell; the later sweeps use 10. Reload-path medians beat recomputation throughout that pilot grid, so it did not locate a crossover length. The next phase changes the axis to simulated bandwidth and includes short prefixes, multiple model families, separate file legs, and repeated sessions. It contains 26,040 accepted full-sweep timings, 0 invalid full-sweep timings, and 10 sweep sessions across 7 models.

The transfer phase uses L4 and Modal's Ampere allocation tier. The observed boards differ: A10 has an active PCIe generation-4 x16 link, while A10G has generation-4 x8. We use A10 calibration, transferred unchanged to A10 and A10G. Their spread is confounded by board, host, and link differences; each board has one main-model session. T4 has 3 main-model sessions and L4 has 2, so their observed ranges do not establish relative platform stability. H100 contributes only complete cells from an interrupted optional GPU instance.

All measured weights and caches use fp16. The transfer runs pin Torch `2.11.0+cu130` and Transformers `4.51.3` to the T4 redesign versions. Python patch versions and GPU drivers differ between platforms. The efficient/math SDPA selection is fixed, without FlashAttention. Source hashes protect the timing, forward, continuation, transfer, and file-read routines against changes from the T4 reference. Platform wrappers change GPU checks, scheduling, and persistence, not those protected routines.

3.2 Models, workloads, and coverage

Table 1 lists the measured checkpoints and verified cache sizes. The formula for fp16 KV bytes per token counts keys and values, layers, KV heads, head dimension, and bytes per element. Configuration sizes are checked against the tensors.

Checkpoint	Layout	KV bytes/token
`Qwen/Qwen2.5-0.5B`	GQA	12,288
`Qwen/Qwen2.5-1.5B`	GQA	28,672
`Qwen/Qwen2.5-3B`	GQA	36,864
`EleutherAI/pythia-1.4b`	MHA	196,608
`facebook/opt-1.3b`	MHA	196,608
`microsoft/phi-2`	MHA	327,680
`unsloth/Llama-3.2-3B-Instruct`	GQA	114,688

Table 1: Released fp16 models and verified KV payload. The Llama checkpoint is the ungated mirror. These are different models, not a controlled attention-layout intervention.

The main model uses a frozen synthetic UltraChat fixture, with conversations stitched for long contexts. A new turn is a fixed-length token continuation of clipped role-labelled text, not necessarily a complete message at a dialogue boundary. Repeated text supplies auxiliary-model controls. Neither workload establishes quality or a natural return distribution.

The transfer main-model grid has prefixes 16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384, with new turns 32 and 256. Controls use a 32-token turn and a smaller repeated-text grid, including native-supported limits where feasible. A10 has main-model measurements only; A10G and L4 also have the controls. Context and memory exclusions remain explicit. Appendix A gives the full completed coverage, including T4-specific reductions and the incomplete H100 subset. We do not replace missing cells with predicted measurements.

3.3 Paths and simulated bandwidth

The reference measures fresh recomputation, a GPU-resident hit, pinned CPU reload, pageable CPU reload, and local file API paths. The raw staged path reads bytes, constructs a CPU tensor view, and copies to GPU. The direct torch path includes deserialization in `torch.load`. Independent microbenchmarks measure H2D movement and file CPU legs without the new turn. Cache creation, CPU offload, packing, pinning, file writes, tokenization, input-ID placement, request queueing, and network transport are outside returning-turn TTFT.

For the *simulated-bandwidth* paths, a packed fp16 buffer is copied in chunks. Each chunk is synchronized, and a cumulative rate limiter waits for its byte deadline. Targets are 50, 100, 200, 400, 800, 1,600 MB/s, plus an unthrottled control. MB/s uses decimal bytes. We record target and achieved rates separately. Buffer creation participates in the pacing budget, and tensor-view work remains inside TTFT; therefore adding an unthrottled overhead intercept is an approximation, especially for short prefixes. This simulates serialized blocking transfer, not physical devices, network latency distributions, or asynchronous pipelines. The unthrottled pinned path is the real pinned-memory control and is counted once.

3.4 Timing, calibration, and uncertainty

Full redesign and transfer sweeps use 10 measured randomized rounds after warmup. Conditions, including prefix-only prefill, are shuffled within each round. CUDA synchronization and events accompany synchronized wall-clock timing; events can include device idle time during host pacing. The endpoint is first-token selection available to the host. Restored-cache continuation consumes the transferred tensors and is checked against the resident-cache continuation and full-recompute first token. This guards against timing a copy that is never used.

Reported within-session intervals use a percentile bootstrap with 10,000 resamples and 95% coverage. Decision scoring uses a paired-round bootstrap interval for reload-minus-recompute TTFT. We call a decision resolved when that interval identifies one path as faster. Intervals covering zero are excluded from the accuracy denominator. Median absolute TTFT error includes complete baseline and reload cells. Related cells share models, contexts, calibration, and sessions; decision counts are descriptive, not independent Bernoulli trials.

Before each new GPU sweep, calibration measures prefix-only, recompute, and resident-hit baselines at a few lengths, plus independent transfer and file legs. It uses 3 baseline rounds and 5 microbenchmark repeats. Nonnegative quadratic time models predict prefill and recompute; resident-hit and byte-cost legs use nonnegative affine fits. Predictions for every planned cell are committed before its sweep begins. Sweep observations never update coefficients. We distinguish anchors, interpolation, and extrapolation; Appendix B.1 specifies anchors and path costs.

Separate session medians are retained. Session spread is the range of those medians divided by their median, not a population interval. Telemetry samples SM and memory clocks, temperature, power, and event reasons at about one sample per second; Modal also records PCIe generation and width. Sampling can miss brief trials, and associations with clocks do not establish a latency cause.

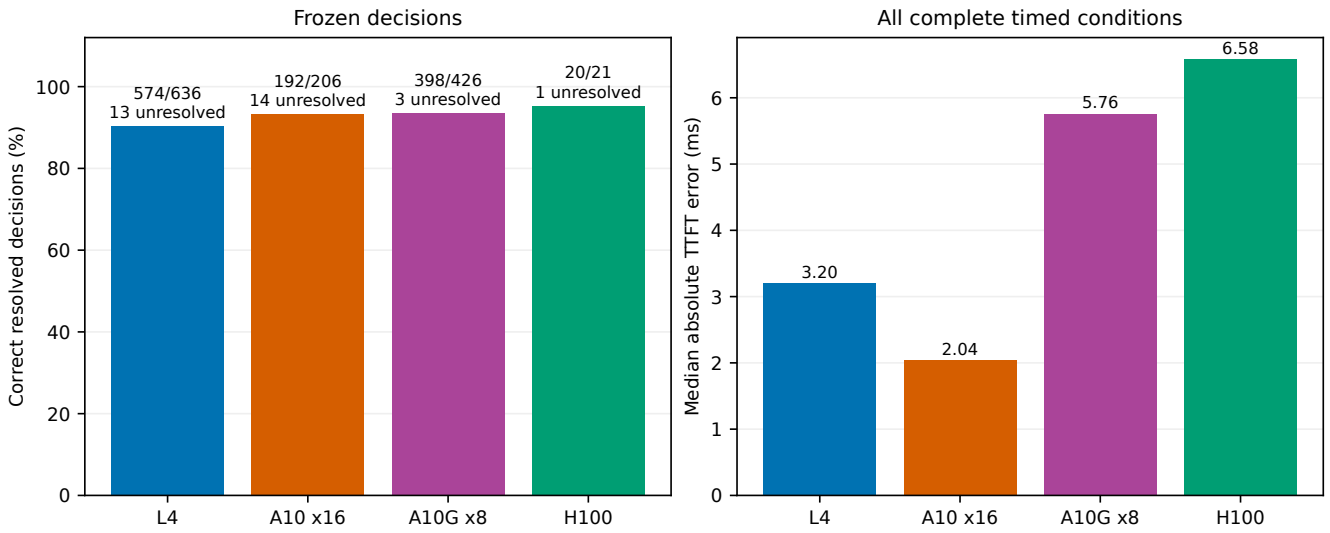


Figure 1: Frozen decisions and median absolute TTFT errors. Scored decisions compare resident hits and reload paths with recomputation; error includes all complete timed conditions. Coverage is unequal: partial H100 has 26 cells, L4 767, A10 x16 260, and A10G x8 507. A10 covers only the main model. Unresolved decisions are excluded. These bars are not a GPU ranking; Appendix B separates coverage and calibration regions.

3.5 File-path observability

Kaggle file reads were physical according to process I/O counters: 7,762/7,762 recorded file observations have positive `physical_read_bytes`, ranging from 196,608 to 1,879,052,288 bytes. These counts include calibration and file microbenchmarks. The counters support reads beyond kernel page-cache hits; they are not physical-device traces and do not identify underlying media or host/storage caches.

Physical reads were not observed on Modal: 0/4,575 file observations have positive physical-read counters. We call this the *local file API path (physical reads not observed)*. Page-cache residency is one interpretation of the zero counters; the cache layer is unverified. Global cache dropping was denied on Kaggle, and per-file eviction advice is advisory on both platforms. Without a cached-versus-uncached ablation, these platforms do not provide equivalent physical-file-read conditions.

4 Results

4.1 Short calibration predicts most decisions on new GPUs

Short calibration predicted most reload-versus-recompute decisions correctly on the tested GPUs. Figure 1 shows 574/636 correct resolved decisions on L4, 192/206 on A10 x16, and 398/426 on A10G x8. Their median absolute TTFT errors are 3.203, 2.040, and 5.758 ms, with 13, 14, and 3 unresolved decisions. H100 contributes 20/21 correct resolved decisions and 1 unresolved decision. Predictions were frozen before these sweeps. A10 covers the main model only, L4 and A10G also cover the controls, and H100 is partial.

4.2 Faster GPUs need more bandwidth before reload wins

The faster GPUs required higher simulated bandwidth before reloading the same prefix became faster than recomputing it. At main-model prefix 1,024 and new turn 32, the T4 sessions cross between target rates of 100–200 or 200–400 MB/s. L4 and A10 x16 cross between 800 and 1,600 MB/s; A10G x8 crosses between 400 and 800 MB/s. H100 still favors recomputation at the highest tested rate. Figure 2 shows the individual achieved-rate brackets. These comparisons use Qwen2.5-1.5B on UltraChat.

4.3 Returning-turn and fixed costs change the answer

Including continuation and fixed costs explains decisions that the prefix-only rule misses. On T4, the adjusted rule agrees with 1,237/1,258 resolved simulated-rate decisions, compared with 1,017/1,258 for the prefix-only rule. For Qwen2.5-0.5B at prefix 512 and turn 32, the prefix-only threshold is 226.626 MB/s, while the adjusted threshold

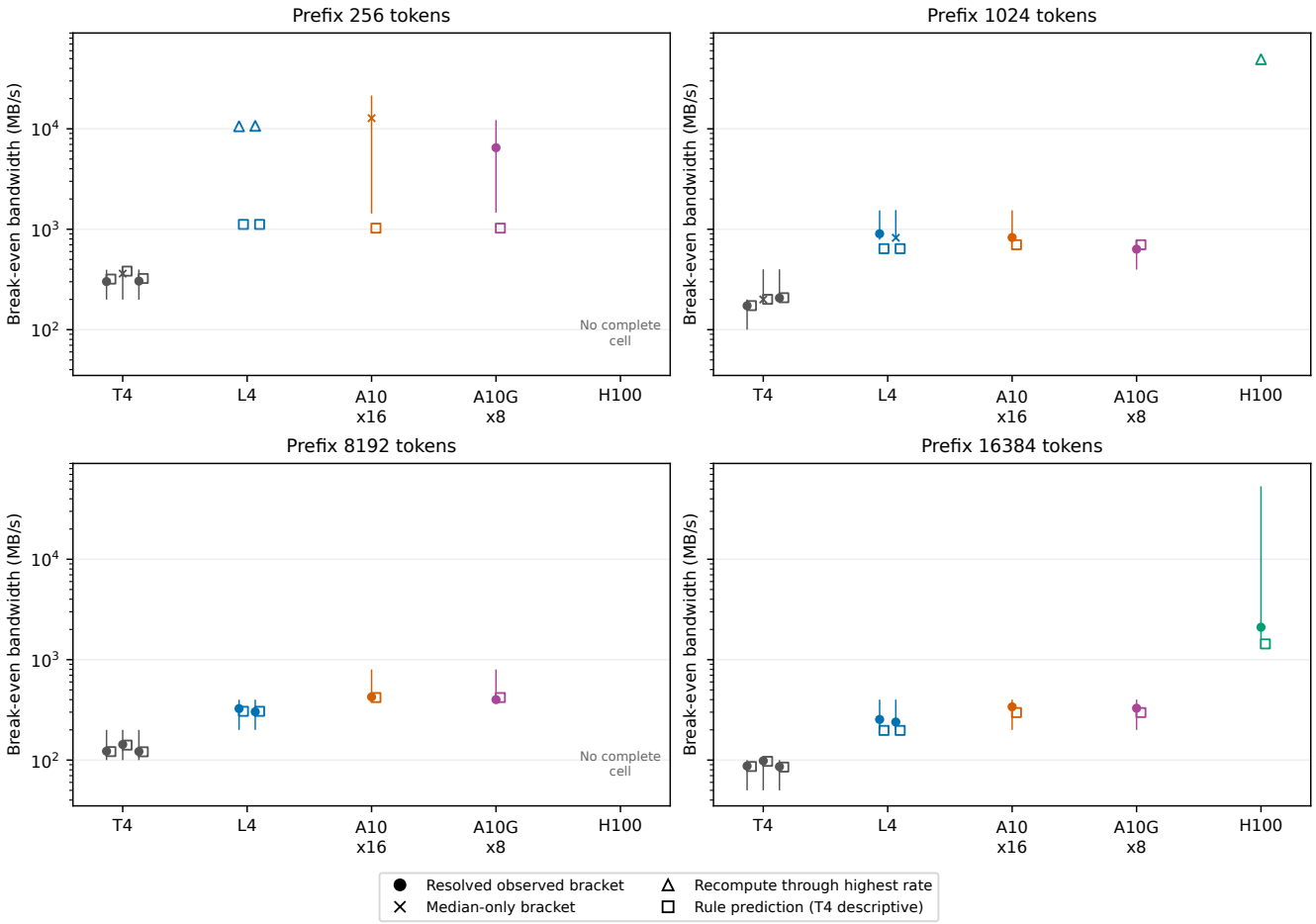


Figure 2: Main-model UltraChat simulated-bandwidth crossovers at matched prefixes, new turn 32. Bars span achieved-rate endpoints. Circles have resolved endpoints; crosses have an unresolved endpoint. Upward triangles mean recompute faster through the highest tested rate; crossover not located. Squares show the rule, descriptive on T4 and frozen on new GPUs. Sessions remain separate.

is 2,442.265 MB/s; the observed crossing lies between the highest paced rate and the unthrottled control. Fixed first-session calibration predicts later T4 main-model sessions with 148/149 and 146/150 correct resolved decisions. The rule comparison uses same-session baselines; the later-session check holds the first session's calibration fixed.

4.4 L4 extends the short-context range where recompute wins

Recomputation remained faster over more of the sampled short-prefix range on L4 than on T4. For a 32-token turn, both L4 sessions favor recomputation at every common prefix in 16, 64, 128, 256. T4 favors reload at prefix 256 in each session, while smaller prefixes include recompute wins and unresolved comparisons. Figure 3 shows the sessions for both turn lengths. These results compare Qwen2.5-1.5B on UltraChat with the real unthrottled pinned-copy path.

4.5 Supporting model and platform observations

At matched repeated-text prefix 512 and turn 32 on T4, Pythia's achieved simulated crossover bracket is $[798.9, 1597.2]$ MB/s, with an unresolved endpoint, versus $[199.5, 398.4]$ for Qwen2.5-1.5B. Phi-2's bracket is $[799.2, 1598.4]$, versus $[99.9, 199.7]$ for Qwen2.5-3B. These selected MHA models need higher simulated bandwidth than the selected GQA models. Different parameter counts, prefill kernels, and widths confound an architectural interpretation; we do not estimate a causal effect of GQA.

Including deserialization reduces the T4 torch-file rule's median absolute relative residual from 11.78% to 3.097%. The raw-format residual changes only from 3.107% to 3.086%. The correction matters to the relevant CPU path rather than improving every format equally. This residual analysis uses independent file legs and same-session

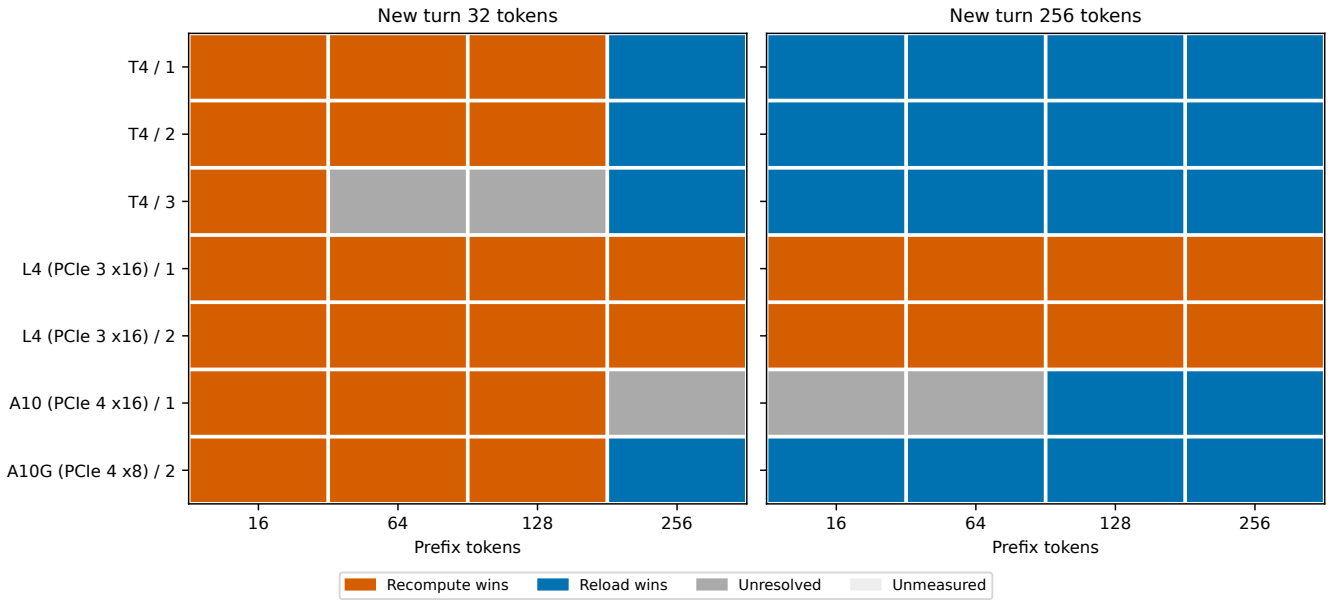


Figure 3: Main-model UltraChat choices between recomputation and the real unthrottled pinned-copy path. Rows are separate sessions; panels separate new-turn lengths. Colors follow the paired timing comparison, with unresolved cells shown explicitly.

baselines; it is not a new frozen-transfer test.

Recompute session spread also matters. Median relative ranges across lengths are 12.341% and 14.847% on T4 for the short and long turn, versus 4.956% and 4.417% on L4. The A10/A10G relative ranges are 23.611% and 22.664%; the board and link differences are described in Section 3.

5 What does not hold

Long-context extrapolation. The low median TTFT errors conceal large native-limit failures. For Qwen2.5-3B at prefix 32,736 with turn 32, A10G predicts 8,041.096 ms recompute but observes 13,914.555 ms, an absolute error of 5,873.459 ms. L4 predicts 13,088.360 ms and observes 18,022.902 ms, an error of 4,934.542 ms. A short nonnegative quadratic calibration is therefore not a reliable native-context TTFT predictor for these observations. High decision agreement at other lengths cannot justify that extrapolation.

Exact proportional throughput scaling. Figure 4 relates the observed simulated-bandwidth boundary to prefix throughput. The frozen throughput-ratio prediction falls inside at least one matched T4 crossover-ratio bracket for 25/27 ratio-resolved L4 shapes, 11/12 A10 shapes, and 20/24 A10G shapes. The partial H100 comparison is 0/1. These are broad-bracket compatibility counts, not tests proving equality. Thresholds above the highest tested rate or with unresolved endpoints do not supply exact ratios. The adjusted denominator also changes with continuation and fixed costs. Dividing two prefix-only formula estimates would merely restate an identity, not validate hardware transfer.

Uniform MHA file outcomes. On repeated-text, matched prefixes 64, 256, 512, 1,024 and a 32-token turn, Pythia's direct-file path has no resolved reload wins on T4, but wins at a matched prefix on A10G. Phi-2 already has direct-file wins on T4, so a blanket assertion that all MHA file paths lose there is false. Both measured MHA models lose on both L4 file paths. A10 x16 has no MHA sweep. These outcomes do not transfer uniformly, and Section 3.5's different read conditions prevent attributing the change to GPU speed alone.

The pilot anomaly remains unexplained. The pilot's prefix-only median was slower than prefix-plus-turn at the long pilot context. None of the 4 phase-two anomaly comparisons has a positive interval lower endpoint for that reversal. The historical cause was not established. Non-reproduction and randomized ordering improve interpretation, but do not retroactively identify an implementation, clock, or thermal cause.

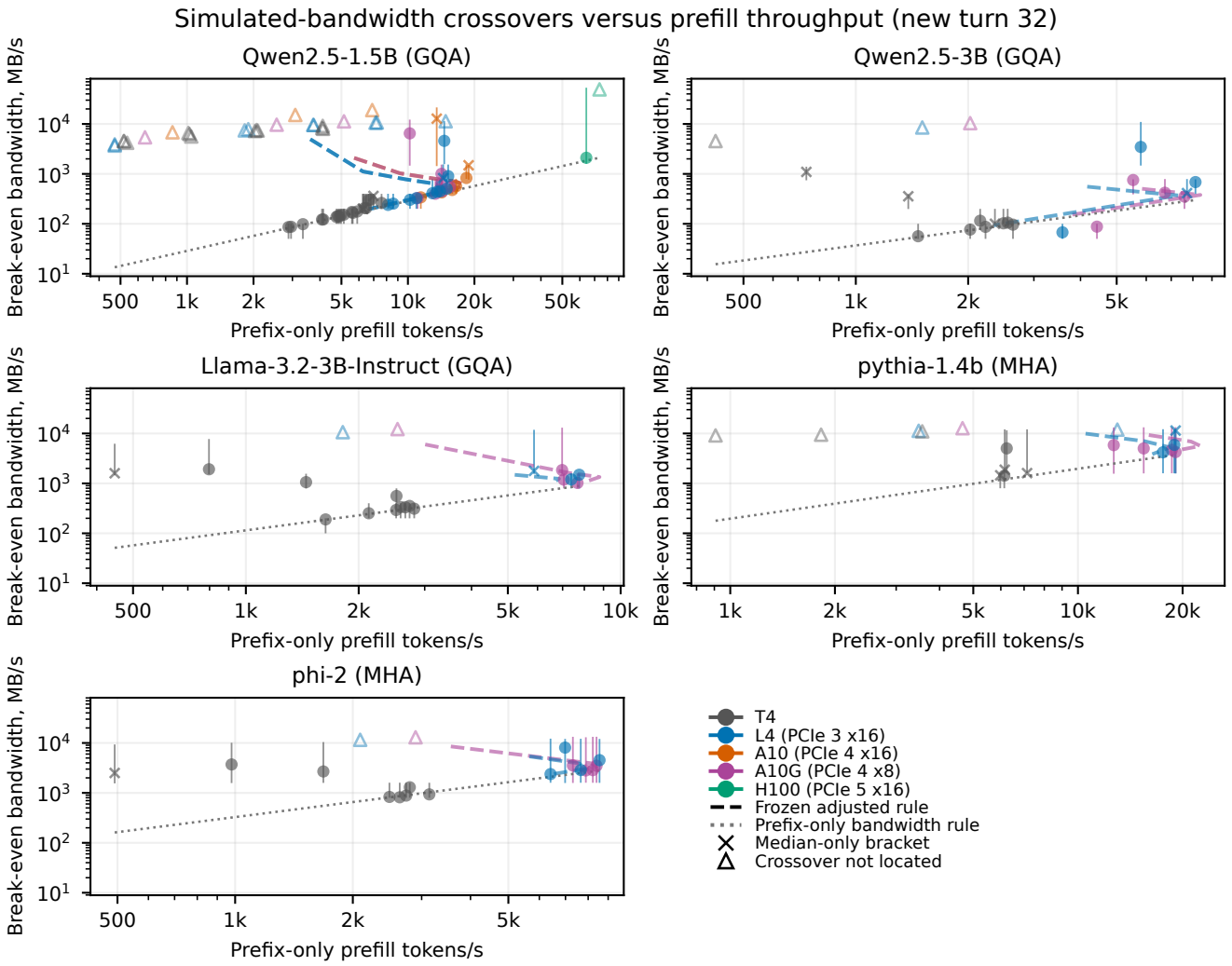


Figure 4: Simulated-bandwidth crossovers and predictions, new turn 32, five models. Observations use measured prefill throughput; dashed adjusted-rule curves use calibration-predicted throughput, so offsets mix throughput and bandwidth prediction errors. T4 is descriptive; other predictions are frozen. Bars span achieved-rate brackets; crosses have an unresolved endpoint. Upward triangles mean recompute faster through the highest tested rate; crossover not located. Downward triangles mean reload faster down to the lowest tested rate; crossover not located. Dotted lines are KV bytes/token times throughput. Interpolated points are not measured thresholds. Model comparisons are confounded; H100 is partial with no MHA sweep.

6 Practical guidance

A one-minute calibration checklist, not a runtime guarantee. After loading and warming the model, measure recompute and resident-hit TTFT at short, typical, and long contexts for the intended turn. Measure pinned/pageable H2D and the file path's CPU read/deserialization legs; check physical-read counters. Apply Equation 2, counting each leg once, and validate close predictions in paired rounds. Recalibrate after changes to the GPU, link, model, dtype, turn, or file path.

Calibration function times of 42.5 seconds on A10 and 63.4 seconds on L4 exclude provider initialization; they do not guarantee this checklist's duration. Near-zero time saved by reuse, untested long contexts, or unknown file-cache state call for more measurement.

The checklist is guidance, not an evaluated deployment. Engine builders must remeasure their actual critical path: buffer preparation, layout, asynchronous copies, and scheduling change it. Overlap scheduling and persistence admission need additional cost models.

7 Related work

LMCache provides external KV storage and engine connectors [4]. vLLM supports prefix reuse and offload [8, 9], while SGLang HiCache adds storage tiers and prefetch [7]. These systems already serve repeated-prefix and multi-turn use.

CacheGen compresses and streams KV [5]. CacheBlend selectively recomputes cached chunks to recover cross-chunk attention [11]. We evaluate uncompressed KV for identical prefixes.

Cake compares GPUs with simulated I/O bandwidth on MHA and GQA models, and overlaps computation from the prefix's beginning with loading from its end [2]. Our rule compares load-only and recompute-only endpoints; it does not determine Cake's optimal partial split. We did not test overlap.

py-kvcache already performs offline per-node/model calibration (its Section 6.6) and presents break-even bandwidth frontiers (Section 5.2), alongside hardware-dependent characterization, asynchronous fetching, and scheduler-aware preloading [3]. Our distinction is timestamped, frozen per-cell predictions evaluated on separate sweeps after short calibration, with commodity/older GPUs, short contexts, and separate CPU-memory and file paths with read observability. Calibration itself is not new; sweep results do not update our predictions.

He et al. distinguish buffered reads from physical-read evidence in observational deployment snapshots [1]. KVServe adapts compression to service conditions [6]; ShadowServe addresses fetching interference [10]. These reinforce costs beyond bandwidth. We adopt no external performance numbers or causal interpretation of host-file differences.

8 Limitations and conclusion

Calibration observes baselines and independent cost legs, not combined reload TTFT. Baseline-fit errors, overhead approximation, and GPU-instance variation therefore remain mixed. The new-GPU tests use local calibration; they are not zero-shot predictions from T4 coefficients. The T4 same-session rule counts are descriptive, while the held-out session check keeps the first session's calibration fixed.

The bandwidth shift and short-context extension describe the observed GPUs rather than an isolated compute effect: host paths, drivers, and links also change. One L4 bandwidth bracket has an unresolved endpoint. A10 and A10G differ at the short-context boundary, and the sampled prefix grid does not locate a continuous crossover length. Decision counts share models, contexts, and calibration, so they are not independent trials. A high aggregate score cannot justify unmeasured contexts. The slower Ampere session did not have lower sampled SM clocks, so those samples alone do not explain its recompute time.

The single-request fp16 reference path establishes neither production-engine performance, multi-request throughput, quantization, bf16, P100 transfer, nor application quality. Production attempts yielded no successful performance observation. Synthetic or repeated text and stitched long inputs do not model a natural return distribution.

Coverage is unequal across boards and model families. H100 has only 26 complete timing cells and 52 unfinished registered cells; only main-model prefixes 1,024 and 16,384 are complete, and its Pythia sweep did not start. Partial records remain in the release but are not scored. One H100 GPU instance cannot establish a general datacenter-GPU or MHA conclusion. Native and memory limits exclude other contexts, and the sparse simulated-rate grid often gives a wide bracket to the unthrottled endpoint.

Frozen-fit uncertainty is not propagated. Within-session intervals and few sessions do not describe a deployment population. File counters identify neither media nor all host caches. Serialized copies do not predict overlap or serving interference. Model differences confound architecture; GPU-instance differences confound causal comparisons.

Within the measured scope, local calibration predicts most resolved decisions. Operators should measure complete returning-turn paths, retain unresolved outcomes, and recalibrate after changes.

References

[1] Ying He, Dingsen Shi, Yanbo Dai, Donger Chen, Harry Fu, Chris Tsu, Wang Feng, and Song Fu. Characterizing Predictability-Latency Trade-offs of KV-Cache SSD Offloading in LMCache for LLM Serving Systems. arXiv:2610.21249v1, 2026. https://arxiv.org/abs/2610.21249v1.

[2] Shuowei Jin, Xueshen Liu, Qingzhao Zhang, and Z. Morley Mao. Compute Or Load KV Cache? Why Not Both? arXiv:2410.03065v2, 2025. https://arxiv.org/abs/2410.03065v2.

[3] Joseph Kanichai, Tiziano De Matteis, and Animesh Trivedi. Building py-kvcache: A Performance Characterization of External KV Caching for vLLM with NVMe SSDs. arXiv:2609.11744v1, 2026. https://arxiv.org/abs/2609.11744v1.

[4] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, and Junchen Jiang. LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. arXiv:2510.09665v2, 2025. https://arxiv.org/abs/2510.09665v2.

[5] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. arXiv:2310.07240v6, 2024. https://arxiv.org/abs/2310.07240v6.

[6] Zedong Liu, Xinyang Ma, Dejun Luo, Hairui Zhao, Bing Lu, Wenjing Huang, Yida Gu, Xingchen Liu, Zheng Wei, Jinyang Liu, Dingwen Tao, and Guangming Tan. KVServe: Service-Aware KV Cache Compression for Communication-Efficient Disaggregated LLM Serving. arXiv:2605.13734v1, 2026. https://arxiv.org/abs/2605.13734v1.

[7] SGLang: HiCache System Design and Optimization. Official project documentation. No individual authors or publication date supplied; accessed November 3, 2026. https://docs.sglang.io/docs/advanced_features/hicache_design.

[8] vLLM: Automatic Prefix Caching. Official project documentation. No individual authors or publication date supplied; accessed November 3, 2026. https://raw.githubusercontent.com/vllm-project/vllm/main/docs/features/automatic_prefix_caching.md.

[9] vLLM: KV Offloading Usage Guide. Official project documentation. No individual authors or publication date supplied; accessed November 3, 2026. https://raw.githubusercontent.com/vllm-project/vllm/main/docs/features/kv_offloading_usage.md.

[10] Xingyu Xiang, Raj Joshi, Yuhan Liu, Jiayi Yao, Chenxingyu Zhao, Junchen Jiang, Yang Zhou, Eddie Kohler, and Minlan Yu. ShadowServe: Interference-Free KV Cache Fetching for Distributed Prefix Caching. arXiv:2509.16857v1, 2025. https://arxiv.org/abs/2509.16857v1.

[11] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. arXiv:2405.16444v3, 2025. https://arxiv.org/abs/2405.16444v3.

A Full completed coverage

The tables are generated from the immutable release, not a new analysis fit. Full checkpoint identifiers and KV sizes appear in Table 1. Phase labels distinguish the pilot, T4 redesign, and transfer. Native-context and budget exclusions are retained; no table entry implies an unmeasured comparison. A10 is PCIe x16, A10G is x8, and their rows are separate.

Table 2: Complete measured coverage. U is UltraChat; R is repeated text. Session counts separate GPU instances, not repeat rounds. Calibration and incomplete H100 cells are excluded.

Phase/board	Model	Text	Turn	Prefix tokens	Sessions
T4	pythia-1.4b	R	32	16, 32, 64, 128, 256, 512, 1,024, 1,792	1
T4	pythia-1.4b	R	256	16, 32, 64, 128, 256, 512, 1,024, 1,792	1
T4	Qwen 0.5B	R	32	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192	1
T4	Qwen 0.5B	R	256	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192	1
T4	Qwen 1.5B	R	32	16, 32, 64, 128, 256, 512, 1,024, 1,792, 2,048, 8,192	1
T4	Qwen 1.5B	U	32	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	3
T4	Qwen 1.5B	U	256	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	3
T4	Qwen 3B	R	32	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
T4	Qwen 3B	R	256	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
T4	opt-1.3b	R	32	16, 32, 64, 128, 256, 512, 1,024	1
T4	opt-1.3b	R	256	16, 32, 64, 128, 256, 512, 1,024	1
T4	phi-2	R	32	16, 32, 64, 128, 256, 512, 1,024, 1,792	1
T4	phi-2	R	256	16, 32, 64, 128, 256, 512, 1,024, 1,792	1
T4	Llama 3.2-3B	R	32	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
T4	Llama 3.2-3B	R	256	16, 32, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
A10	Qwen 1.5B	U	32	16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
A10	Qwen 1.5B	U	256	16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
A10G	pythia-1.4b	R	32	64, 256, 512, 1,024, 2,016	1
A10G	Qwen 1.5B	U	32	16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
A10G	Qwen 1.5B	U	256	16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	1
A10G	Qwen 3B	R	32	64, 256, 512, 1,024, 32,736	1
A10G	phi-2	R	32	64, 256, 512, 1,024, 2,016	1
A10G	Llama 3.2-3B	R	32	64, 256, 512, 1,024	1
H100	Qwen 1.5B	U	32	1,024, 16,384	1
L4	pythia-1.4b	R	32	64, 256, 512, 1,024, 2,016	1
L4	Qwen 1.5B	U	32	16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	2
L4	Qwen 1.5B	U	256	16, 64, 128, 256, 512, 1,024, 2,048, 4,096, 8,192, 16,384	2
L4	Qwen 3B	R	32	64, 256, 512, 1,024, 32,736	1
L4	phi-2	R	32	64, 256, 512, 1,024, 2,016	1
L4	Llama 3.2-3B	R	32	64, 256, 512, 1,024	1
Pilot	Qwen 1.5B	R	32	512, 2,048, 8,192	1
Pilot	Qwen 1.5B	R	256	512, 2,048, 8,192	1

B Per-board frozen scorecards

The release committed the Ampere predictions under its API tier name before the sweeps: A10 calibration, transferred unchanged to A10 and A10G. The presentation separates observed boards without changing those predictions. “Controls” cover the repeated-text auxiliary models; main sessions use UltraChat. Anchor, interpolation and extrapolation refer to the calibration length range, not observed bandwidth precision. Error includes baseline conditions; correct/resolved counts concern the resident hit and reload decisions.

Table 3: Frozen scores by observed board and calibration region or session. Error is median absolute TTFT error in milliseconds, over all complete predicted conditions. Decisions exclude unresolved paired intervals.

Board/link	Scope	Correct/resolved	Unresolved	Cells	Error
L4 x16	all	574 / 636	13	767	3.203
L4 x16	anchor	246 / 248	5	299	2.822
L4 x16	interpolate	231 / 289	8	351	2.981
L4 x16	extrapolate	97 / 99	0	117	4.007
L4 x16	l4-controls	193 / 201	8	247	1.506
L4 x16	l4-main-1	188 / 216	4	260	3.681
L4 x16	l4-main-2	193 / 219	1	260	3.654
A10 x16	all	192 / 206	14	260	2.040
A10 x16	anchor	59 / 63	3	78	2.275
A10 x16	interpolate	96 / 105	5	130	2.308
A10 x16	extrapolate	37 / 38	6	52	1.341
A10 x16	a10-main-1	192 / 206	14	260	2.040
A10G x8	all	398 / 426	3	507	5.758
A10G x8	anchor	178 / 187	0	221	5.094
A10G x8	interpolate	170 / 185	2	221	5.511
A10G x8	extrapolate	50 / 54	1	65	8.371
A10G x8	a10-controls	193 / 209	0	247	5.031
A10G x8	a10-main-2	205 / 217	3	260	6.433
H100 x16	all	20 / 21	1	26	6.579
H100 x16	anchor	11 / 11	0	13	0.332
H100 x16	extrapolate	9 / 10	1	13	9.999
H100 x16	h100-main	20 / 21	1	26	6.579

B.1 Calibration anchors and path costs

The frozen registrations use the following prefix anchors for prefix-only, recompute, and resident-hit baselines. L4 and A10 use the same anchors; H100 uses them for its calibrated Qwen2.5-1.5B and Pythia models, although its Pythia sweep did not start.

Model	Prefix anchors (tokens)
Qwen2.5-1.5B and Qwen2.5-3B	64, 1,024, 8,192
Llama-3.2-3B-Instruct	64, 512, 1,024
Pythia-1.4B and Phi-2	64, 512, 2,016

Prefix-only and recompute times use a three-parameter quadratic fit with nonnegative coefficients, with length scaled by the largest anchor. Resident-hit times and byte-cost legs use nonnegative affine fits. The pinned-transfer fit's intercept supplies fixed overhead for simulated-bandwidth predictions; the unthrottled path uses that fit's full predicted transfer time.

Each file prediction adds resident-hit time to independently measured legs. The raw staged path adds raw read, tensor-view construction, and H2D transfer. The T4 staged torch path adds raw read, deserialization, and H2D transfer. Direct torch loading adds the CPU `torch.load` leg and H2D transfer. Each leg includes its own measured or fitted overhead; the pinned-transfer intercept is not added again. T4 file residuals use same-session baselines; frozen transfer predictions use calibration fits. The registrations retain every anchor, coefficient, and per-cell prediction. A "new turn" is a fixed-length token continuation of clipped role-labelled text, rather than a guaranteed dialogue boundary.

C Reproduction without new experiments

The archive, claim inventory, and figure sources are part of `v0.1.0-data`. The manuscript preserves the scorecard and regime PDFs. The GPU-bandwidth figure changes legend wording; the throughput figure changes panel layout and legend wording. Neither changes data or predictions. `NUMBERS.md` lists source fields, row predicates, aggregations, rounding, and all uses. Its audit script checks provenance and coverage, rather than recomputing every statistic.

C.1 Data reproduction

These commands reconstruct the released analysis on CPU:

```
git clone https://github.com/mohi-devhub/reload-or-recompute.git
cd reload-or-recompute
git checkout v0.1.0-data
python3 -m venv .venv
source .venv/bin/activate
python3 -m pip install -r requirements-analysis.txt
python3 scripts/materialize_results.py
python3 scripts/analyze_phase2.py --from-results outputs/results.jsonl
python3 scripts/report_phase2.py
python3 scripts/analyze_phase3.py --from-results outputs/results.jsonl
python3 scripts/audit_phase3.py
python3 scripts/report_phase3.py
python3 scripts/consolidate_study.py --check --figures
```

C.2 Manuscript reproduction

The data tag has no manuscript directory. After data reproduction, check out the separate manuscript tag `paper-v0.1.1` and compile its self-contained sources:

```
git checkout paper-v0.1.1
cd paper
pdflatex -interaction=nonstopmode -halt-on-error main.tex
bibtex main
pdflatex -interaction=nonstopmode -halt-on-error main.tex
pdflatex -interaction=nonstopmode -halt-on-error main.tex
```

Citation verification records distinguish publication metadata from experiment measurements. New source checks correct bibliography metadata only; they add no experimental results.